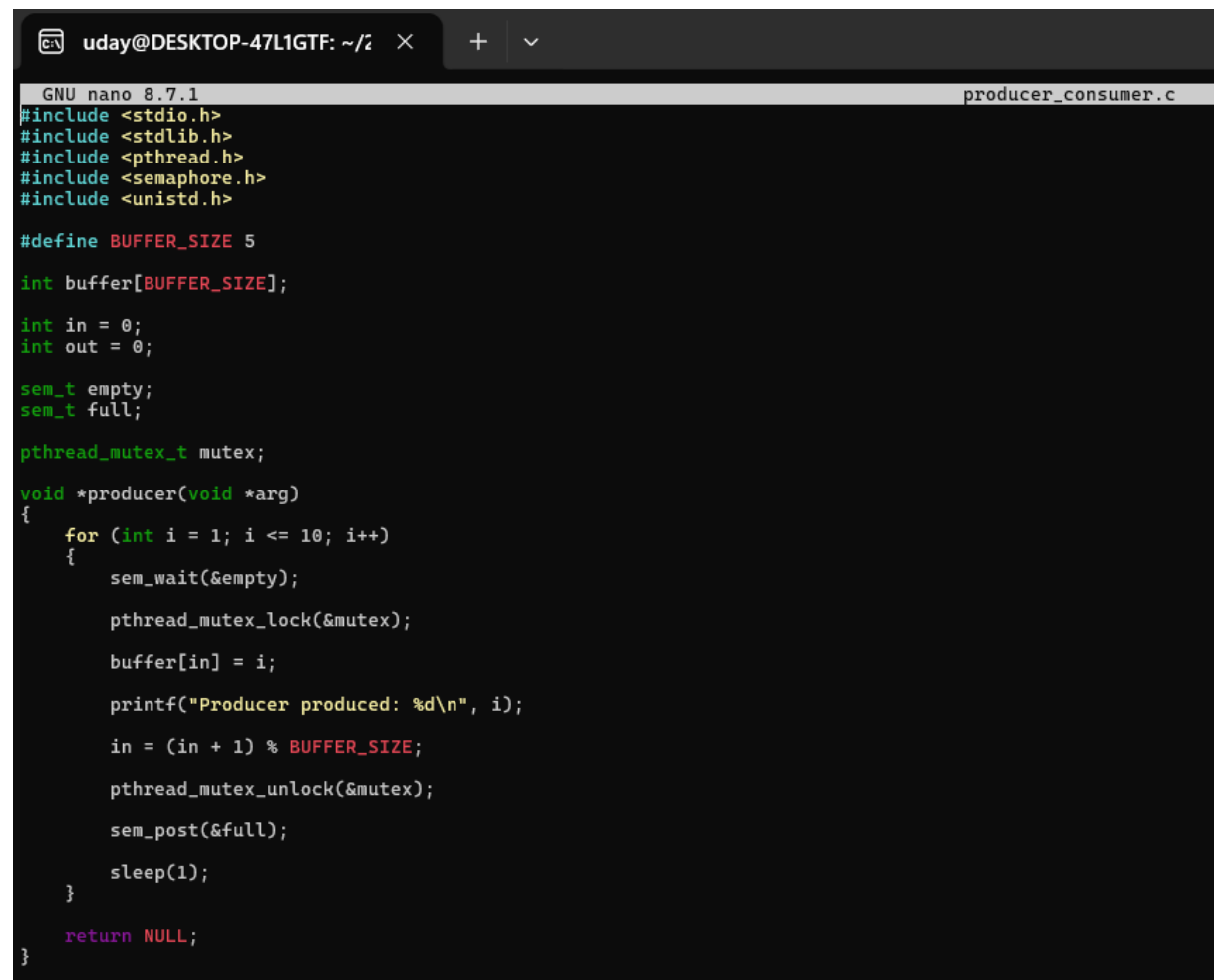


Practical -12

1) Producer consumer



```
GNU nano 8.7.1 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= 10; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;

        printf("Producer produced: %d\n", i);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        sleep(1);
    }

    return NULL;
}
```

```
uday@DESKTOP-47L1GTF: ~/z × + v
GNU nano 8.7.1

    pthread_mutex_lock(&mutex);
    int item = buffer[out];
    printf("Consumer consumed: %d\n", item);
    out = (out + 1) % BUFFER_SIZE;
    pthread_mutex_unlock(&mutex);
    sem_post(&empty);
    sleep(2);
}
return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

```
uday@DESKTOP-47L1GTF: ~/2520090089_OSSP/2520090089_Practical/Practical-12$ ./producer_consumer
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Producer produced: 4
Consumer consumed: 3
Producer produced: 5
Producer produced: 6
Consumer consumed: 4
Producer produced: 7
Producer produced: 8
Consumer consumed: 5
Producer produced: 9
Producer produced: 10
Consumer consumed: 6
Consumer consumed: 7
Consumer consumed: 8
Consumer consumed: 9
Consumer consumed: 10
```

2) Deadlock

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}
```

```
u day@DESKTOP-47L1GTF: ~/2 × + v
GNU nano 8.7.1
printf("Thread 1: acquired both resources\n");

pthread_mutex_unlock(&resource2);
pthread_mutex_unlock(&resource1);

return NULL;
}

void *thread2(void *arg)
{
printf("Thread 2: locking Resource 2\n");
pthread_mutex_lock(&resource2);

sleep(1);

printf("Thread 2: waiting for Resource 1\n");
pthread_mutex_lock(&resource1);

printf("Thread 2: acquired both resources\n");

pthread_mutex_unlock(&resource1);
pthread_mutex_unlock(&resource2);

return NULL;
}

int main()
{
pthread_t t1, t2;

pthread_mutex_init(&resource1, NULL);
pthread_mutex_init(&resource2, NULL);

pthread_create(&t1, NULL, thread1, NULL);
pthread_create(&t2, NULL, thread2, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

return 0;
}
```

```
u day@DESKTOP-47L1GTF: ~/2520090089_OSSP/2520090089_Practical/Practical-12$ ./deadlock
Thread 1: locking Resource 1
Thread 2: locking Resource 2
Thread 1: waiting for Resource 2
Thread 2: waiting for Resource 1
^C
```

3) Deadlock prevention

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
```

```
u day@DESKTOP-47L1GTF: ~/2 X + v
GNU nano 8.7.1
printf("Thread 1: acquired both resources\n");

pthread_mutex_unlock(&resource2);
pthread_mutex_unlock(&resource1);

return NULL;
}

void *thread2(void *arg)
{
printf("Thread 2: locking Resource 1\n");
pthread_mutex_lock(&resource1);

sleep(1);

printf("Thread 2: locking Resource 2\n");
pthread_mutex_lock(&resource2);

printf("Thread 2: acquired both resources\n");

pthread_mutex_unlock(&resource2);
pthread_mutex_unlock(&resource1);

return NULL;
}

int main()
{
pthread_t t1, t2;

pthread_mutex_init(&resource1, NULL);
pthread_mutex_init(&resource2, NULL);

pthread_create(&t1, NULL, thread1, NULL);
pthread_create(&t2, NULL, thread2, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

pthread_mutex_destroy(&resource1);
pthread_mutex_destroy(&resource2);

return 0;
}

u day@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Practical-12$ ./deadlock_prevention
Thread 1: locking Resource 1
Thread 2: locking Resource 1
Thread 1: locking Resource 2
Thread 1: acquired both resources
Thread 2: locking Resource 2
Thread 2: acquired both resources
```



uday@DESKTOP-47L1GTF: ~/z



GNU nano 8.7.1

```
        pthread_mutex_lock(&mutex);

        int item = buffer[out];

        printf("Consumer consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        sleep(2);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```